CS 342 – OPERATING SYSTEMS

PROJECT 2 – REPORT

Mehmet Efe Mutlu	Emir Said Bakan	Ahmet Eren Gökalp
22303326	22302852	22302136

Introduction

This report aims to evaluate the behavior and performance of the implemented library through several experiments. This report focuses on measuring the overhead of thread creation, joining, yielding, and scheduler choice under different workloads through visualized tables and plots.

Experimental Setup

All experiments were performed on a Linux x86-64 environment consistent with the project requirements. The library was implemented in C, and the scheduling policy was selected through tus_init () as either FCFS or RANDOM. Since the system is cooperative, threads only give up the CPU when they call tus_yield (), so the number of yield calls directly affects execution behavior.

Each created thread was given a 16 KB stack, and the total number of simultaneously existing threads was kept within the project limit of 64, including the main thread. Execution time was measured around each workload, and experiments were repeated multiple times to obtain more reliable results. The tests were designed to examine the effects of thread count, number of yields, and scheduling policy on total runtime.

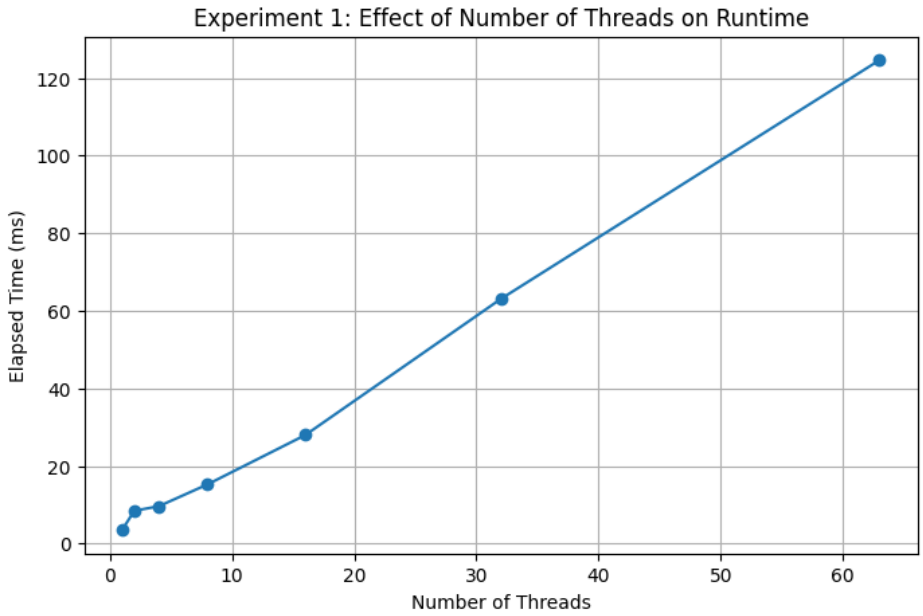
Experiment 1: Effect of Number of Threads

The purpose of this experiment was to observe how the total runtime of the TUS library changes as the number of created threads increases. Increasing the number of threads is expected to increase the overhead of thread creation, scheduling, context switching, and joining.

In this experiment, the scheduling algorithm is fixed to FCFS. Each thread executed the same workload by calling the tus_yield () function 5000 times and then terminating. The number of threads is varied as 1, 2, 4, 8, 16, 32, and 63. The elapsed time was measured from before thread creation until after all threads had been joined, so the result includes thread creation, cooperative context switching, scheduling, and cleanup overhead. Since the library is cooperative, context switches occurred only when threads explicitly yielded.

Number of Threads and It's Effects on Measured Time (ms)

Number of Threads	Iterations per Thread	Total Yields	Measured Time (ms)
1	5000	5000	3.503
2	5000	10000	8.450
4	5000	20000	9.612
8	5000	40000	15.266
16	5000	80000	27.973
32	5000	160000	63.047
63	5000	315000	124.601



The results show that the runtime generally increased as the number of threads increased. This is expected because more threads require more stack allocation, more context initialization, more ready-queue operations, and more joins at the end. In addition, keeping the per-thread workload fixed means that the total number of yields also increases with thread count, which further increases runtime. Overall, the experiment shows that the performance cost of the TUS library grows as the number of active threads increases.

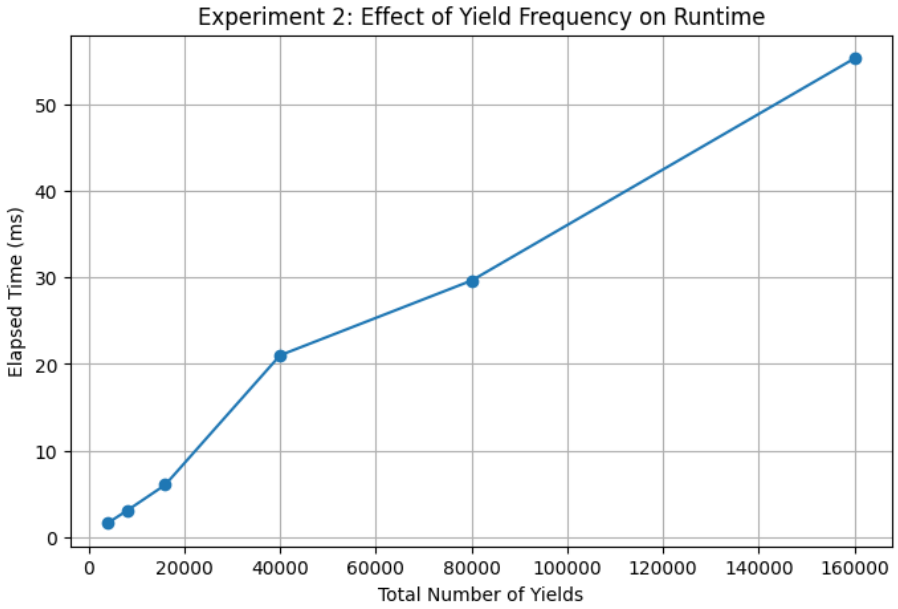
Experiment 2: Effect of Number of Yields

The purpose of this experiment was to observe how the total runtime changes as the number of cooperative yield operations increases. Context switching only happens when a thread explicitly calls tus_yield (). Therefore, increasing the number of yields directly increases the number of scheduling decisions and context switches performed by the library.

In this experiment, the number of threads was fixed at 8, and the scheduling algorithm was fixed to FCFS. Each thread executed the same code, but the number of iterations is varied as 500, 1000, 2000, 5000, 10000, and 20000. In each iteration, the thread called the tus_yield function so increasing the iteration count increased the total number of cooperative context switches. The elapsed time was measured from before thread creation until after all threads had been joined.

Number of Yields and It's Effects on Measured Time (ms)

Number of Threads	Iterations Per Thread	Total Yields	Measured Time (ms)
8	500	4000	1.647
8	1000	8000	3.071
8	2000	16000	6.053
8	5000	40000	21.008
8	10000	80000	29.634
8	20000	160000	55.281



The results show that runtime increased as the number of yields increased. This is expected because every tus_yield() requires the library to save the current thread context, choose the next runnable thread, and restore another context. As the number of yield operations grows, these scheduling and context-switching costs accumulate and lead to longer total execution time. Overall, this experiment shows that yield frequency has a direct effect on the performance of the TUS library.

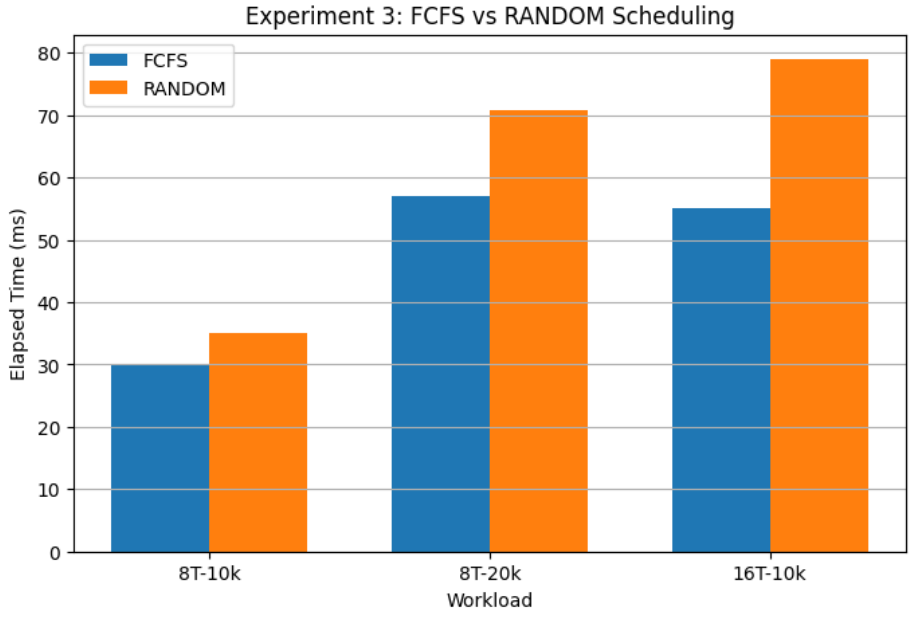
Experiment 3: Effect of Scheduling Algorithm

This experiment compared the runtime of the two scheduling algorithms implemented in TUS library: FCFS and RANDOM. The goal was to see how scheduler choice affects performance under the same workload. The scheduler is invoked whenever a thread calls tus_yield().

Three workloads were tested. In the first, 8 threads each performed 10000 iterations. In the second, 8 threads each performed 20000 iterations. In the third, 16 threads each performed 10000 iterations. Each workload was run once with FCFS and once with RANDOM, and the elapsed runtime was measured.

Scheduling Algorithm and It's Effects on Measured Time (ms)

Scheduling Algorithm	Number of Threads	Iterations per Thread	Total Yields	Measured Time (ms)
FCFS	8	10000	80000	30.040
RANDOM	8	10000	80000	35.067
FCFS	8	20000	160000	56.981
RANDOM	8	20000	160000	70.828
FCFS	16	10000	160000	55.111
RANDOM	16	10000	160000	78.981



The results show that FCFS was faster than RANDOM in all cases. The difference became larger as the workload increased, which suggests that RANDOM introduces more scheduler overhead. Overall, this

experiment shows that FCFS provides lower runtime overhead and more predictable performance than RANDOM for these workloads.

Conclusion

This report evaluated the performance of the implemented TUS library under different workloads and scheduling policies. The experiments showed that runtime increased as the number of threads increased and as the number of cooperative yield operations increased. This was expected because larger workloads require more thread creation, context switching, scheduling, and join operations. The comparison of scheduling algorithms also showed that FCFS consistently performed better than RANDOM for the tested cases, while RANDOM introduced additional overhead due to its less predictable thread selection. Overall, the results confirm that the implemented library behaves as expected and demonstrate the main performance characteristics of cooperative user-level threading.